a) Realice la elección de un diseño para esta realidad.

Debe indicar los Tipos Abstractos de Datos elegidos, por ejemplo:

Vehículos = Secuencia (Vehículo)

Vehículo = matricula X marca X año

Auto = cant_puertas X precio_base X Vehículo

Moto = cilindrada X precio_base X Vehículo

Observaciones:

-
-
-
-

b) Realice el correspondiente desglose de requerimientos.

Requerimiento 1a: Alta de vehículo.

Entrada: Datos del vehículo

Salida: Vehículo ingresado

Proceso:

Si matricula ya existe entonces

Error: La matricula ya existe

Sino

Ingreso los datos del vehiculo al sistema

Requerimiento 1b: Baja de vehículo.

Entrada:

Salida:

Proceso:

Requerimiento 2: Calcular suma de precios de alquiler diario.

Entrada:

Salida:

Proceso:

Requerimiento 3: Obtener vehículos con mayor y menor precio de alquiler diario.

Entrada:

Salida:

Proceso:

Requerimiento 4: Contar motos por cilindrada.

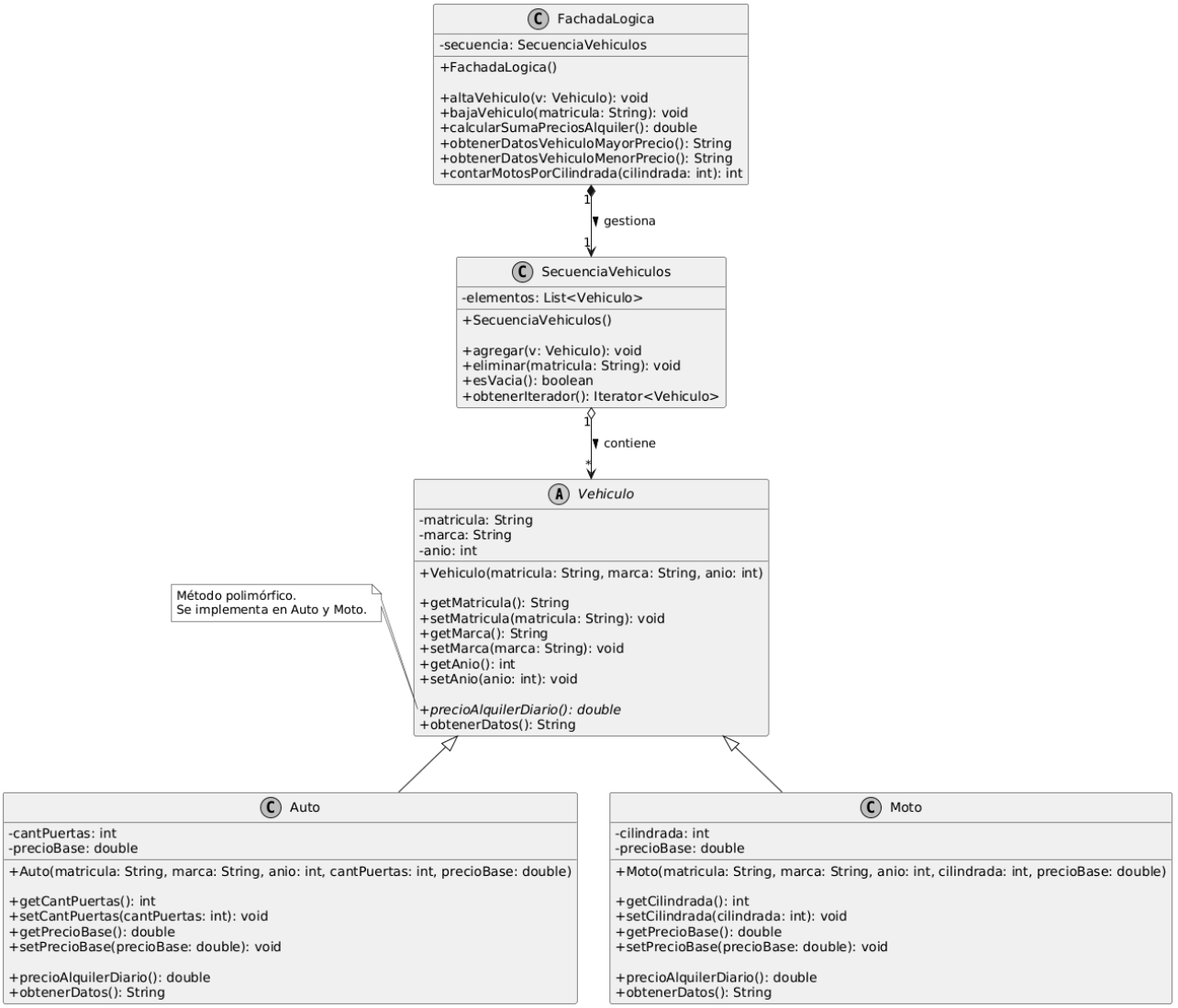
Entrada:

Salida:

Proceso:

c) En base a las partes anteriores, realice la especificación del diseño elegido.

Debe presentar el **Diagrama de Clases de Implementación en UML**, mostrando solamente los atributos, y luego detallar la especificación de cada clase por separado, incluyendo:


FachadaLogica
-secuencia: SecuenciaVehiculos
+FachadaLogica()
+altaVehiculo(v: Vehiculo): void
+bajaVehiculo(matricula: String): void
+calcularSumaPreciosAlquiler(): double
+obtenerDatosVehiculoMayorPrecio(): String
+obtenerDatosVehiculoMenorPrecio(): String
+contarMotosPorCilindrada(cilindrada: int): int
1
gestiona
1
SecuenciaVehiculos
-elementos: List<Vehiculo>
+SecuenciaVehiculos()
+agregar(v: Vehiculo): void
+eliminar(matricula: String): void
+esVacia(): boolean
+obtenerIterador(): Iterator<Vehiculo>
1
contiene
*
Vehiculo
-matricula: String
-marca: String
-anio: int
+Vehiculo(matricula: String, marca: String, anio: int)
+getMatricula(): String
+setMatricula(matricula: String): void
+getMarca(): String
+setMarca(marca: String): void
+getAnio(): int
+setAnio(anio: int): void
+precioAlquilerDiario(): double
+obtenerDatos(): String
Método polimórfico.
Se implementa en Auto y Moto.
Auto
-cantPuertas: int
-precioBase: double
+Auto(matricula: String, marca: String, anio: int, cantPuertas: int, precioBase: double)
+getCantPuertas(): int
+setCantPuertas(cantPuertas: int): void
+getPrecioBase(): double
+setPrecioBase(precioBase: double): void
+precioAlquilerDiario(): double
+obtenerDatos(): String
Moto
-cilindrada: int
-precioBase: double
+Moto(matricula: String, marca: String, anio: int, cilindrada: int, precioBase: double)
+getCilindrada(): int
+setCilindrada(cilindrada: int): void
+getPrecioBase(): double
+setPrecioBase(precioBase: double): void
+precioAlquilerDiario(): double
+obtenerDatos(): String

-